

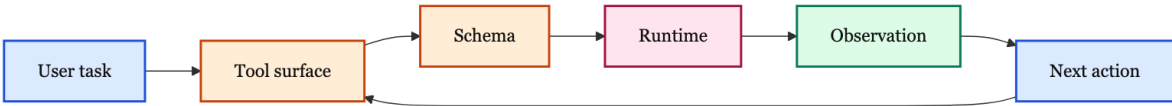
第 20 讲: Tool Design for Agents: 让工具成为稳定行动空间

Coding Agent Notes (June 2026)

系统位置	本讲位于 ACI、harness、context engineering 和 trajectory integrity 之后，专门解释 agent 能够行动之前，工具面本身应该怎样设计。
已知前提	已经理解 coding agent 的 action/observation loop，也知道 trajectory 必须能被记录、验证和回放。
新增问题	为什么 tool schema 不是普通 API 文档；怎样让模型选对工具、填对参数、理解结果、从错误中恢复，并在权限边界内行动。
读完可回答	一个 search/edit/run/delegate 工具集怎样命名、怎样拆分粒度、怎样返回 observation、怎样暴露副作用，以及怎样通过 transcript 评测迭代。

1 核心图景

Tool design 的核心问题不是“给模型几个函数”。对 agent 来说，工具定义就是行动空间：它告诉模型有什么外部动作可做、每个动作在什么条件下应该被调用、参数怎样构造、runtime 怎样执行、结果怎样解释，以及失败后能否恢复。普通 API 文档主要面向确定性程序员；agent tool schema 面向的是一个会推理、会猜测、会犯错、会在上下文限制中选择行动的非确定性系统。



这张图给出了本讲的主线。用户任务不会直接变成外部行为，而是先经过 tool surface：哪些工具在当前 turn 可见，哪些工具被 namespace 隐藏，哪些工具需要 search 或 approval。模型随后根据 schema 构造调用，runtime 验证和执行，observation 把下一步需要的证据返回给模型。下一步行动再回到同一个工具面里继续选择。工具设计的好坏，就体现在这个 loop 是否稳定、低歧义、可恢复、可审计。

中心命题:

一个好 tool 不是“把后端函数暴露出来”，而是把外部系统整理成 agent 能理解、能选择、能验证、能恢复的行动空间。
如果工具名、参数、结果、错误和权限边界没有形成一致契约，模型能力越强，越可能把错误更快地放大。

这也是为什么本讲不把 tool design 写成 SDK 使用说明。SDK 提供的是语法：名字、描述、输入 schema、输出 schema、timeout、approval 和 conditional enabling。系统设计要回答更高一层的问题：这些语法如何组合成一个适合 agent 的行动接口。

2 API wrapper 为什么不等于 agent tool

很多系统的第一版工具面，都是把已有后端 API endpoint 包一层函数调用。例如后端有 list_users、get_user、list_transactions、create_ticket，就原样给模型四个工具。这个做法实现快，但通常不是 agent-friendly 的设计。

原因在于 API endpoint 面向的是确定性程序：程序员知道调用顺序，知道中间对象在哪里，知道哪个 ID 是后续调用需要的，知道怎样处理分页、空结果、权限和重复请求。Agent 面对的是自然语言

任务和有限上下文，它要在工具描述里推断“我现在该用哪个动作”。如果工具过细，模型要在每一步承担过多选择成本；如果工具过粗，runtime 又无法验证和约束每一步。

设计方式	表面上解决了什么	实际风险
原样 wrapper	快速把 API 暴露给模型	工具过多、语义重叠、上下文浪费、调用顺序错误
单个 mega-tool	模型只需选一个入口	参数语义巨大、行为不透明、错误难定位、权限难隔离
workflow tool	面向任务拆分动作	需要理解用户 workflow，并为 observation 和错误设计契约

Anthropic 的工具设计经验里有一个很好的判断：不要只问“后端有哪些函数”，而要问“agent 在真实任务中自然会需要哪些动作”。搜索联系人通常不应该是 `list_contacts`，而应该是 `search_contacts`；查日志通常不应该把所有日志塞回上下文，而应该是 `search_logs` 并返回相关行和上下文；处理客户问题也不一定要让模型先查客户、再查交易、再查备注，而可以提供 `get_customer_context` 类型的聚合工具。

Insight. API wrapper 的危险在于它把后端的分层泄漏给模型。Agent tool 应该暴露任务空间的自然动作，而不是数据库、微服务或 SDK 的历史边界。

3 工具面的第一原则：少、清楚、可动态加载

工具越多，不一定能力越强。工具定义会进入模型上下文，占用 token，也增加选择空间。OpenAI 的 function calling 指南明确建议初始可用函数保持较小，并在大型工具生态里用 tool search 或 namespace 延迟加载。Anthropic 的经验也相同：过多或重叠工具会分散 agent 的策略选择。



这个图里的每一层都在控制“当前模型到底看见什么”。小的 initial set 让常用动作随手可用；namespace 让相关工具组成有边界的组；tool search 让模型在需要时加载一部分工具；conditional enable 根据用户权限、运行环境、任务状态隐藏不可用工具；approval gate 则把高风险动作交给用户或系统确认。

初始工具面要服务当前任务，而不是展示系统能力。

如果模型一开始就看到几十个相似工具，它要把上下文花在比较工具描述上，而不是理解任务。一个 coding agent 的常驻工具面通常应该覆盖少数高频动作：读文件、搜索、编辑、运行命令、汇报状态。低频或高风险动作，例如发布、删除、迁移数据库、访问外部账号，可以动态加载或需要 approval。

Namespace 的作用是降低选择熵。

Namespace 不是装饰性的前缀，而是模型的工具地图。比如 `repo.search`、`repo.read`、`repo.edit` 与 `github.issue_read`、`github.pr_comment` 应该表达不同系统边界。如果两个 namespace 里的工具都叫 `search`，description 必须说明搜索对象、返回形式和适用任务。否则模型会在“看起来都能搜索”的工具之间随机选择。

动态工具面必须可解释。

当工具被条件启用或延迟加载时，系统要能解释为什么某些工具不可见。不可见工具不应该被模型猜测；可见工具不应该需要用户推断权限。对于生产系统，tool visibility 本身也是安全和调试信号：某个失败是模型没选对工具，还是工具因为权限、环境、feature flag、budget 被隐藏了。

4 命名：先让模型选对工具

工具名是模型选择动作时看到的第一个强信号。一个名字如果只反映内部实现，模型就需要从 description 里补全部语义；一个名字如果反映任务意图，模型可以更早缩小搜索范围。MCP 规范里把 name、title 和 description 分开，是很有意义的：name 是程序化标识，title 面向 UI 和人类理解，description 则像给模型的使用说明。

元素	应该表达	常见失败
name	稳定、短、可路由的动作标识	过度缩写、内部代号、同义工具重名
title	人类可读的展示名	和 name 完全重复，不能帮助审阅
description	何时用、何时不用、参数含义、输出含义	只写一句“calls API”，没有边界和例外
参数名	业务语义和数据形状	用 user 代替 user_id，用 data 代替具体字段

好的命名通常有三个特征。第一，动词清楚：search_logs 比 logs 好，apply_patch 比 patch 好。第二，对象清楚：search_repo_symbols 和 search_web 不应该都叫 search。第三，副作用清楚：preview_migration 和 run_migration 必须分开，draft_email 和 send_email 不能混在同一个模糊工具里。

命名还要配合评测。Anthropic 特别提醒，prefix-based 和 suffix-based namespace 对不同模型可能有不同效果。结论不是“某个命名永远最优”，而是工具名属于可测变量：如果 transcripts 里频繁出现错选工具，就要把命名、description 和 namespace 一起放进 eval loop。

5 参数 schema：让无效状态不可表示

参数 schema 的目标不是让 JSON 看起来规范，而是尽量让错误调用在进入 runtime 前就暴露出来。OpenAI 的 best practice 里有一句很强的原则：用 enum 和对象结构让 invalid states unrepresentable。Coding agent 的工具尤其需要这个原则，因为一个错误参数可能不是简单失败，而是写错文件、运行错目录、覆盖用户修改或污染训练轨迹。

schema 技术	解决的问题	coding-agent 例子
enum / Literal required fields	限制动作集合 避免缺关键参数	file editor 的 command enum edit 必须有路径，insert 必须有行号和文本
range / pattern object structure strict mode	约束数值和字符串形状 避免互斥参数共存 阻止模型临时编造字段	timeout 非负，score 在 0 到 100 search query 与 filters 分层表达 closed schema + required fields

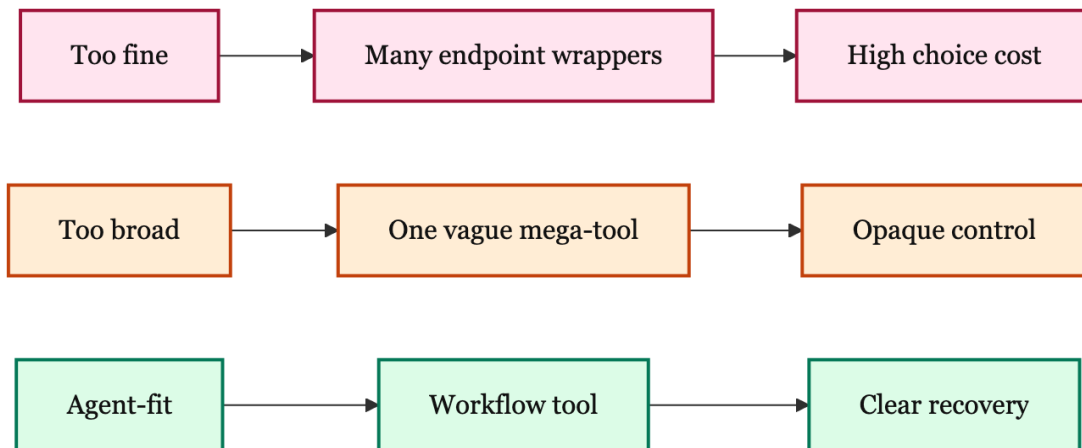
OpenHands 的 file editor 是一个具体例子。它没有让模型传一个自由形态的“edit instruction”，而是把 command 限定为 view/create/str_replace/insert/undo_edit，把 path 要求为绝对路径，把 old_str、new_str、insert_line、view_range 分别标明使用条件。这个工具的 description 还强调 exact matching 和 uniqueness：old_str 必须精确匹配文件中唯一的一段文本。这里的 schema 和 description 共同把“编辑”变成可验证的动作，而不是让模型自由描述想改哪里。

Insight. 工具参数越接近自然语言愿望，runtime 越难保护系统；工具参数越接近可验证操作，模型越需要 schema 帮它少犯错。好设计是在两者之间找到 agent 能用、系统能验的边界。

严格 schema 也有成本。所有字段都 required、additionalProperties=false 会降低模型乱填字段的概率，但也可能让可选信息变得笨重。因此工程上常见做法是：对高风险动作强 schema，对探索性只读动作允许更宽松的 query；对用户已知、系统已知的参数，不让模型重复填写，而由 runtime 注入。例如当前 workspace、用户 ID、会话 ID、selected file 这些信息，很多时候不该成为模型参数。

6 工具粒度：太细、太粗、刚好

粒度是 tool design 里最容易失衡的部分。太细的工具让模型在每一步都做低价值决策；太粗的工具把太多控制权藏到 runtime 里，使 observation、权限和错误不可解释。合适的粒度应该围绕“一个 agent 可以理解并验证的子任务”来切。



太细的工具类似把所有 endpoint 都暴露出来。模型要决定先 list 还是 get，拿哪个 ID，翻几页，怎样合并结果，怎样处理空值。太粗的工具则像 `do_everything(task)`：看起来简单，但任何失败都只能返回“没完成”，很难知道是检索失败、权限失败、编辑失败，还是验证失败。Agent-fit 的 workflow tool 会把稳定的机械步骤放进代码，把需要模型判断的地方留在 schema 和 observation 上。

场景	建议粒度	原因
搜索文件内容 读取超大文件	<code>grep(pattern,path,include)</code> <code>read(path,range)</code>	模型决定 query，工具处理遍历和排序 模型控制范围，工具避免整文件塞入上下文
编辑代码	<code>str_replace/insert</code>	修改点可验证，diff 可观察
运行命令	<code>terminal(command,timeout)</code>	保留 shell 表达力，但暴露超时和状态
复杂子任务	<code>agent_as_tool(input)</code>	用结构化输入和 max turns 限定边界

粒度的判断标准不是“函数多还是少”，而是失败能否被定位、调用能否被审计、结果能否指导下一步。如果一个工具每次都被另一个工具紧跟着调用，可以考虑合并；如果一个工具内部可能产生多个高风险副作用，就应该拆开并加入 approval 或 preview。

7 Observation: 返回下一步行动需要的证据

Observation 是工具设计中最容易被低估的部分。很多系统把 tool output 当作字符串返回，能让模型看到结果就算完成。但对 agent 来说，observation 是下一步行动的输入，也是 trajectory 的证据。一个好的 observation 不只是“结果文本”，还要说明调用发生了什么、状态是否改变、是否截断、下一步该如何恢复。

MCP 的 structuredContent 和 outputSchema 给了一个清楚方向：结构化结果可以被 schema 验证，也能让客户端和 LLM 更可靠地解析。Anthropic 的经验则强调返回 high-signal context：少给低层技术 ID，多给语义名称；需要下游调用时再提供必要 ID；长结果要分页、过滤、范围选择或截断，并明确告诉模型怎样缩小查询。

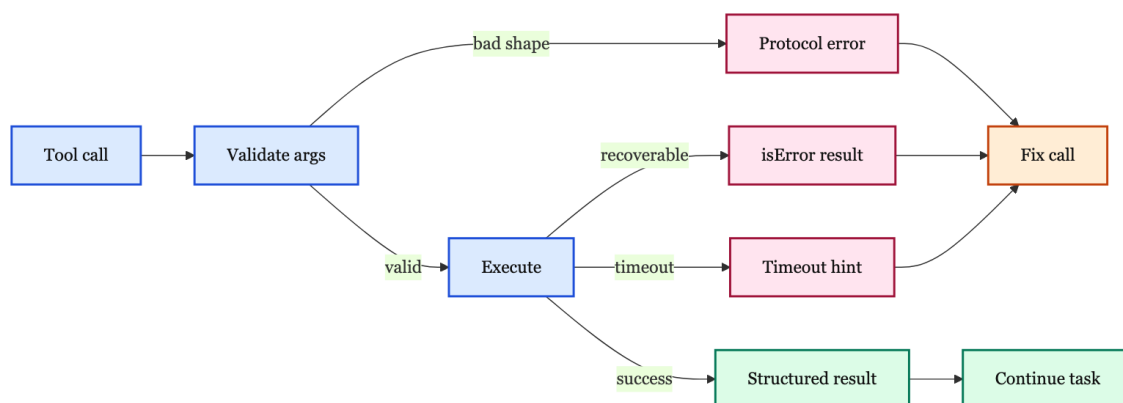
Observation 字段	价值	例子
状态摘要	让模型知道动作是否完成	exit code、timeout、created/modified
证据片段	支撑下一步判断	matched lines、diff、test failure snippet
结构化 ID	支撑后续调用	file path、command id、issue number
截断提示	防止模型误以为看全了	truncated=true、full output path
恢复建议	把失败转成可行动信息	“narrow query”或“old_str not unique”

OpenHands 的 terminal observation 很典型：它不仅返回 stdout/stderr，还保留 command、exit code、timeout、working directory、Python interpreter、完整输出保存路径，并在输出过长时截断。对 coding agent 来说，exit_code=0 和“输出里看起来没有 error”不是同一件事；当前工作目录也不是装饰信息，而是防止下一步命令跑错位置的状态证据。

File editor observation 则体现了另一类 observation：旧内容、新内容、路径、文件是否存在，以及可视化 diff。它让模型和审阅者都能看见实际修改，而不是只相信模型说“我改好了”。这也和 trajectory integrity 相连：如果轨迹里没有 diff 或状态变化，后续训练和评测很难知道 action 是否真的产生了正确外部后果。

8 Error design: 错误必须可恢复

错误返回不是实现细节。对 agent 来说，错误是下一步策略的一部分。一个好的错误会告诉模型“哪里不满足 schema 或 runtime 条件，以及下一步怎样修复”；一个坏错误只会暴露 stack trace、HTTP code 或含糊的“failed”，让模型猜测。



MCP 把错误分成两类很有用。Protocol error 适合 unknown tool、invalid arguments、server error 等调用层问题；tool execution error 则以 isError=true 出现在 tool result 中，表示工具被正确调用了，

但执行阶段遇到 API 失败、业务逻辑失败或输入数据不满足环境条件。OpenAI Agents SDK 也允许 timeout 或 failure 作为 model-visible result 返回，让 agent 有机会调整参数、缩小范围、换工具或请求用户。

错误类型	适合表达	好的返回方式
schema error	参数缺失、类型错误、未知字段	指出字段和合法格式
not found	文件、资源、ID 不存在	返回搜索范围和相近候选
ambiguous	匹配多处、候选过多	要求增加上下文或 filter
timeout	命令仍在运行或外部服务慢	返回 partial output 和继续/停止选项
permission	需要确认或权限不足	说明缺什么权限，不自动绕过

以 file editor 为例，old_str 不唯一时，坏错误是“replace failed”；好错误应该说明匹配了多处，并建议增加上下文。以 search tool 为例，结果被截断时，坏 observation 是只给前 100 条；好 observation 应该明确 truncated=true，并建议加 path、include 或更具体 pattern。Agent 的恢复能力来自这些可操作信息，而不是来自模型凭空反省。

Insight. 错误消息应该像下一步行动的 affordance。它不是给人看的抱怨，而是给模型的恢复接口。

9 副作用、幂等性和权限

Tool design 必须显式区分读操作、写操作、破坏性操作、幂等操作和 open-world 操作。MCP 的 ToolAnnotations 提供了四个有用提示：是否只读、是否可能破坏、是否幂等、是否会接触开放外部世界。这些字段只是 hint，不能替代真正的权限系统，但它们能帮助客户端决定 UI 提示、确认策略、并发策略和审计策略。

语义	设计含义	coding-agent 例子
read-only	不改变环境，可更自由并行	grep、read file、repo map query
destructive	可能删除、覆盖、迁移、发布	file edit、shell command、deploy
idempotent	重复调用无额外影响	读取、搜索、设置同一配置值
open-world	会接触外部不封闭环境	web、terminal、remote API、email

OpenHands 的三个工具给出了直观对比。Grep 是 read-only、non-destructive、idempotent、closed-world，并且不需要资源锁，因为它只读本地文件、可以并行。File editor 是 destructive、non-idempotent、closed-world，并且要按文件路径声明资源，避免同一文件并发读写。Terminal 是 destructive、non-idempotent、open-world，因为 shell 命令可以修改文件、访问网络、启动长进程，还可能依赖当前会话状态。

权限设计不能只靠模型遵守提示。高风险工具应该有 approval gate、preview、dry-run、scope restriction 和 audit log。比如 send_email 必须和 draft_email 区分；delete_file 必须比 view_file 有更强确认；terminal 应该有 working directory、sandbox、timeout、network policy 和输出记录。模型可以建议动作，但 runtime 必须是边界执行者。

10 Tool surface 的动态管理

动态工具面解决的是规模问题。ToolLLM 和 Gorilla 都说明了一个事实：工具/API 生态一旦变大，不能把所有工具描述都塞进单个上下文。ToolBench 收集了 16,464 个 REST API；Gorilla 讨论的是大量变化、重叠、文档不断更新的 API。这样的规模下，模型首先要解决的不是“怎样调用某个 API”，而是“怎样找到相关工具，并拿到足够新的文档”。

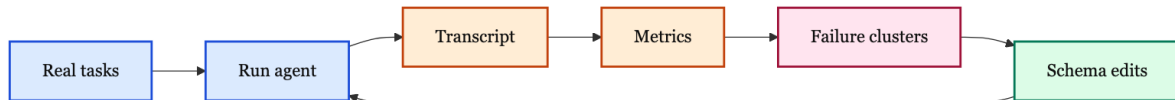
在产品系统里，动态工具面通常有四种机制：

机制	作用	风险
tool search	大工具集按需加载	搜索描述太弱会漏工具
namespace	按领域组织工具	namespace 过大仍然选择困难
conditional enable	根据权限/环境/状态隐藏工具	失败原因不透明
allowed tools	在某 turn 限制动作空间	过窄会阻止合理路径

动态加载不是把复杂性藏起来，而是把复杂性分层。第一层是高层地图：有哪些 namespace，它们负责什么。第二层是具体工具：工具被加载后，description 要足够详细，参数和输出要足够明确。第三层是 runtime policy：哪些工具因为权限、环境、budget、approval 被禁用。好的系统会把这些状态作为可诊断信息，而不是让模型面对一个神秘消失的工具。

11 Tool design 的评测闭环

工具设计不能只靠审查 schema。一个工具看起来合理，不代表模型会按预期使用。Anthropic 的建议非常直接：构造真实任务，运行 agent，收集 runtime、tool calls、token consumption、tool errors，读 transcripts，观察冗余调用、错误参数、无效查询和结果误读。ToolLLM 的 ToolEval 和 Gorilla 的 APIBench 则从研究角度说明，tool-use 能力需要专门评测，而不是普通问答分数。



评测闭环要看至少六类信号。

信号	代表什么	可能改哪里
wrong tool	选择空间或命名有问题	name、namespace、description
invalid args	schema 或参数说明不清	enum、required、examples、Field
redundant calls	粒度太细或结果不够高信号	合并工具、调整 observation
ignored result	observation 不可读或太长	结构化输出、摘要、格式
timeout/error	runtime policy 不适合任务	timeout、pagination、retry hint
unsafe action	权限边界不够硬	approval、allowed tools、sandbox

这个评测闭环也解释了为什么“few-shot demo”不能替代 eval。Demo 可以帮助初始使用，但真实问题会暴露模型如何误解边界。比如模型总是给 search query 自动加年份，说明 description 需要纠正；模型总是先 list all，再自己筛选，说明工具粒度和结果格式在浪费上下文；模型总是重试同一个无效参数，说明错误返回没有给出可恢复 hint。

12 Coding agent 的工具家族

一个 coding agent 的工具面通常可以分成六个家族：read、search、edit、run、inspect state、delegate。每一类工具的 schema 重点不同。

家族	代表工具	设计重点
Read	view file, open doc	range、encoding、truncation、当前路径
Search	grep, symbol search	query、path/include filters、排序、截断
Edit	apply patch, file editor	exact match、diff、undo、资源锁
Run	terminal, tests	working dir、timeout、exit code、long output
State	git status, session info	dirty state、branch、pending process
Delegate	subagent, Codex tool	input schema、max turns、approval、structured output

这些工具不能用同一种模板设计。Search 的结果应该尽量高召回但可筛选；read 的结果应该范围清楚；edit 的结果必须有 diff 和失败原因；terminal 的结果必须有 exit code、timeout 和当前目录；delegate 的结果必须限制任务边界，否则就变成另一个不受控 agent。一个系统如果把这些都统一成 run(command)，表面上最灵活，实际上把所有约束都丢给模型。

13 一个具体重构例子

考虑一个内部客服系统。后端已经有很多 API：列出客户、读取客户、列出订单、读取订单、列出退款、创建备注、发送邮件。最直接的做法是把这些 endpoint 全部包装成工具。这样模型当然“能做事”，但在真实任务里会出现三个问题。第一，模型会先 list 很多对象，再在上下文里人工筛选，浪费 token。第二，模型会把后端 ID 和用户可读名称混在一起，后续调用容易填错。第三，一旦任务失败，很难判断是没找到客户、订单过滤错了、退款状态没看懂，还是最终动作权限不够。

更适合 agent 的第一版工具面可以这样改：

原始 endpoint 思路	Agent-friendly 工具	设计理由
list_customers	search_customer	按名称、邮箱、订单号搜索，避免全量列表
get_customer + 多个 list	get_customer_context	聚合近期订单、退款、备注和风险标记
list_logs	search_payment_logs	返回相关日志行和前后文，不让模型读全量日志
send_email	draft_retention_email	先生成草稿和证据，发送动作另设 approval

这个重构的关键不是把工具数从七个降到四个，而是把 agent 需要思考的部分和代码应该承担的部分重新划线。搜索和聚合是确定性系统更擅长的工作；判断用户为什么离开、怎样解释风险、是否需要人工确认，是模型更适合参与的工作。好的 tool design 会把机械遍历、分页、join、排序和截断放进工具实现，把策略选择、解释和权衡留给模型。

Observation 也要跟着改。一个 get_customer_context 不应该只返回一大段 JSON，而应该返回客户摘要、相关订单、最近异常、可引用证据、是否截断、后续可用 ID。否则模型虽然拿到了“更多数据”，但没有拿到“下一步应该怎么行动的证据”。

Insight. Agent-friendly 工具通常不是后端 API 的一对一映射，而是面向真实 workflow 的可验证子任务。工具内部可以调用多个 endpoint，但对模型暴露的是一个清楚的行动承诺。

14 常见坏味道

Tool design 的问题经常不是单点 bug，而是工具面长期演化后留下的坏味道。看到这些信号时，通常应该回到 transcripts 和 eval 里验证。

工具名像内部缩写。

如果工具叫 `op_v2`、`ctx_fetch`、`svc_exec`，模型只能从 `description` 猜用途。内部缩写对工程团队很熟，对模型和新用户都不友好。工具名应优先表达动作和对象，内部服务名可以放在实现层或 `source card` 里。

description 只描述实现，不描述边界。

“Calls the billing API”不是一个合格 `description`。模型需要知道何时使用、何时不用、输入字段的业务含义、输出字段代表什么、常见错误如何恢复。尤其是 `search`、`edit`、`run` 这类高频工具，`description` 应该显式说明默认范围、排序方式、截断规则和安全限制。

返回结果像数据库 dump。

把所有字段原样返回，通常会降低 agent 表现。低层字段、UUID、内部状态码、完整嵌套对象会挤占上下文，也会制造误读。更好的方式是默认返回 `concise observation`，并提供 `response_format`、`range`、`filter` 或 `follow-up ID`，让模型在需要时再取细节。

错误消息只对开发者友好。

`Stack trace`、`HTTP 500`、`database constraint violation` 对开发者有用，但对模型恢复帮助有限。`Agent-facing error` 要告诉模型下一步怎么改：缩小查询、换路径、补字段、增加上下文、等待长命令、请求权限，或者停止并向用户报告。

所有工具都默认可并行。

并行调用不是纯性能问题。只读搜索可以并行，编辑同一文件不能随便并行，`terminal session` 可能需要串行，外部 API 还可能有 `rate limit`。工具 `schema` 和 `runtime` 需要表达 `resource locks`、`idempotence` 和 `open-world side effects`，否则并发会破坏 `trajectory` 的因果顺序。

15 从单个工具到工具集

单个工具设计正确，并不保证工具集整体正确。Agent 实际面对的是一组可组合动作：先搜索，再读取，再编辑，再运行测试，再根据失败继续搜索。工具之间如果没有共同语义，模型会在组合时出错。比如 `search` 返回相对路径，`read` 要求绝对路径；`edit` 返回 `diff`，但 `run test` 不返回工作目录；`terminal` 允许长进程，`state` 工具却无法告诉模型哪个进程还在运行。这些都不是单个工具 `schema` 的 `bug`，而是工具集契约不一致。

一个工具集至少需要统一四类约定。

组合约定	要统一什么	失败表现
对象命名	<code>path</code> 、 <code>symbol</code> 、 <code>issue</code> 、 <code>command id</code> 的格式	<code>search</code> 找到的对象不能直接 <code>read/edit</code>
状态边界	<code>working directory</code> 、 <code>branch</code> 、 <code>session</code> 、 <code>environment</code>	命令跑错目录或编辑错工作区
证据格式	<code>diff</code> 、 <code>exit code</code> 、 <code>test failure</code> 、 <code>truncation</code>	模型误判“已经验证”
副作用顺序	哪些动作可并行，哪些必须串行	覆盖编辑、重复提交、脏状态丢失

这也是为什么 `coding agent` 不能只做一个强大的 `terminal`。`Terminal` 的表达力最高，但它不天然给出结构化 `object identity`、资源锁、权限含义和恢复 `hint`。一个好的工具集会保留 `terminal` 作为 `escape hatch`，同时为常见任务提供更结构化的 `read/search/edit/test` 工具。这样模型在简单路径上使用受约束工具，在少数无法表达的路径上再进入 `shell`。

工具集设计还要考虑“证据传递”。如果 `search` 找到一个函数，`read` 应该能用同一种路径和 `range` 读取；如果 `edit` 修改了文件，`run test` 应该能明确使用同一个 `working directory`；如果 `test` 失败，`observation` 应该返回 `failure snippet` 和可复现命令；如果最终汇报，模型应该能引用这些证据，而不是只说“我运行了测试”。这条证据链就是 #19 讲过的 `trajectory integrity` 在 `tool design` 层面的前置条件。

Insight. 工具集不是工具的集合，而是一套可组合的行动语言。单词定义得再好，如果语法不一致，agent 仍然写不出稳定的程序。

16 Schema review worksheet

在真正合并一个新工具之前，可以用一张 review worksheet 把设计检查拉回具体问题。这个 worksheet 不依赖某个 SDK，适用于 MCP server、OpenAI function tool、Claude tool、OpenHands tool 或内部自定义 harness。

问题	审查方式
一句话用途	去掉实现细节后，能否用一句话说明这个工具帮 agent 完成什么子任务
使用边界	description 是否写明何时用、何时不用、和相邻工具的差异
输入来源	哪些参数应由模型填写，哪些应由 runtime 从会话状态注入
参数约束	是否能用 enum、range、pattern、required、object structure 减少坏调用
输出证据	observation 是否足以支持下一步行动、最终汇报和失败诊断
错误恢复	每类常见失败是否给出可操作修复，而不是只返回内部错误
副作用	read-only/destructive/idempotent/open-world 是否和真实实现一致
并发边界	是否声明资源锁，是否允许并行，重复调用是否会改变状态
评测任务	是否至少有 5 到 10 个真实任务覆盖正常路径、边界路径和失败路径

这张表的价值在于它迫使设计者把“模型会懂的”假设写成可检查契约。比如“输入来源”这一项经常暴露隐藏问题：如果当前 repo 路径已经在 runtime 里，模型就不应该每次填写 workspace root；如果用户身份来自登录态，模型也不应该编造 user id。把已知参数交给模型填写，会把确定性状态变成生成式猜测。

“输出证据”这一项也很关键。一个工具如果返回“success”，却不返回改了哪个文件、产生了什么 diff、运行了哪个验证命令，后续模型只能相信自己刚才的计划。对人类开发者来说，计划和证据是两件事；对 agent 来说，这个区分更重要，因为它可能会把“我准备运行测试”误写成“测试已经通过”。因此工具结果应该尽量让任务状态从计划语言回到证据语言。

最后，worksheet 不是一次性 checklist，而是 eval 之后要反复更新的文档。每次 transcript 暴露 wrong tool、invalid args、ignored result 或 unsafe action，都应该映射回其中一行：是命名问题、schema 问题、observation 问题、错误恢复问题，还是权限边界问题。这样 tool design 才会随着真实 agent 行为收敛，而不是随着开发者直觉漂移。

17 工程检查表

检查项	合格标准
工具选择	常驻工具少，namespace 清楚，大工具面可 search/defer
命名	name 稳定，title 可读，description 说明何时用和何时不用
参数	JSON Schema/Pydantic 约束明确，invalid states 尽量不可表示
粒度	机械步骤放进工具，策略选择留给模型，失败可定位
Observation	返回状态、证据、截断、结构化字段和恢复信息
错误	区分 schema/protocol/execution/timeout/permission，并给 actionable hint
副作用	read-only/destructive/idempotent/open-world 显式表达，并有权限边界
运行时	timeout、sandbox、working directory、resource locks、audit log 明确
评测	用真实任务 transcripts 度量 wrong tool、invalid args、redundant calls、tool errors

18 最终 takeaway

Tool design 的目标,是把真实世界里复杂、危险、变化快的系统,整理成 agent 可以稳定行动的接口。这个接口不只是函数签名,而是一个完整契约:工具面控制选择空间,命名表达任务边界, schema 限制参数, runtime 执行权限和超时, observation 返回证据, 错误消息提供恢复路径, 评测闭环持续发现工具设计的弱点。

Insight. 模型能力、context engineering 和 harness 都很重要,但 agent 真正接触世界的地方是 tool。工具设计得好,模型的推理会被转化成可验证行动;工具设计得差,模型的推理会被放大成不可控副作用。

因此,设计 coding-agent tools 时要始终问三个问题。第一,模型能不能在当前上下文里选对它。第二,模型能不能构造一个 runtime 可验证的调用。第三,工具返回的 observation 能不能让模型做出更好的下一步。如果这三个问题都答不上来,这个 tool schema 就还不是一个稳定行动空间。

实际落地时,优先级可以很简单。先保证高频只读工具清楚、快速、低噪声,因为它们决定 agent 能否理解环境;再保证编辑和运行工具可验证、可恢复,因为它们决定 agent 能否安全改变环境;最后再扩展 delegation、external API 和 open-world tools,因为这些工具最容易带来权限、成本和不可复现问题。工具设计的成熟度,通常不是看工具数量,而是看每个工具是否让下一步行动更确定。

参考资料

- [1] [OpenAI Function Calling](#). Official documentation.
- [2] [OpenAI Agents SDK Tools](#). Official documentation.
- [3] [Writing effective tools for agents](#). Anthropic Engineering article.
- [4] [MCP Server Tools Specification](#). Protocol specification.
- [5] [MCP Schema Reference](#). ToolAnnotations schema.
- [6] [OpenHands](#). Repository source for coding-agent tools.
- [7] [ToolLLM / ToolBench](#). Tool-use dataset and evaluation paper.
- [8] [Gorilla / APIBench](#). API calling and hallucination evaluation paper.